

# **STUDENT GRADE MANAGEMENT SYSTEM**

## **PROJECT REPORT**

**Programming of Python Essentials Project**

|                         |                               |
|-------------------------|-------------------------------|
| <b>Student Name</b>     | Bhupendra Pratap Singh Sengar |
| <b>Roll Number</b>      | 26BCE11722                    |
| <b>Course</b>           | CSE1021                       |
| <b>Program Language</b> | Python                        |

**Submitted a first-semester Python Essentials Vithyarthi Project**

## 1. Introduction

The Student Grade Management System is a simple Python program for storing student details and marks. It calculates total marks, percentage and grade for a student.

The project is designed as a beginner-level command-line application and uses basic Python programming concepts such as variables, input and output, lists, dictionaries, functions, loops and conditional statements.

## 2. Problem Statement

Calculating total marks, percentage and grade manually can take time when there are several student records. This project provides a simple way to enter student information and perform these calculations automatically.

## 3. Objectives

- Store basic student information.
- Enter marks for different subjects.
- Calculate total marks.
- Calculate percentage.
- Assign a grade according to the percentage.
- Search for a student using the roll number.
- Practice basic Python programming concepts.

## 4. Scope of the Project

The project is intended for basic student grade management. It allows student records to be entered, viewed and searched during the execution of the program. The current version is a command-line program and does not require a graphical interface or database.

## 5. Tools and Technologies

| Item                 | Details                            |
|----------------------|------------------------------------|
| Programming Language | Python 3                           |
| Interface            | Command Line                       |
| External Libraries   | None                               |
| Storage              | In-memory during program execution |

## 6. Subjects

The program accepts marks for five subjects:

- Mathematics
- Programming
- Physics

- English
- Electronics

Each subject is considered out of 100 marks, giving a maximum total of 500 marks.

## 7. Grade Criteria

| Percentage   | Grade |
|--------------|-------|
| 90 and above | S     |
| 80 – 89.99   | A     |
| 70 – 79.99   | B     |
| 60 – 69.99   | C     |
| 50 – 59.99   | D     |
| Below 50     | F     |

## 8. Working of the Project

When the program starts, it displays a menu with options to add a student, view students, search for a student, calculate a student's grade, or exit the program.

**Add Student:** The user enters the roll number, name and marks. The program checks that marks are between 0 and 100 and stores the record.

**View Students:** The program displays the student records stored during the current execution.

**Search Student:** The user enters a roll number and the program searches the stored records.

**Calculate Grade:** The program calculates total marks and percentage and then assigns a grade.

**Exit:** The program ends when the user selects the exit option.

## 9. Formula Used

$$\begin{aligned}\text{Total Marks} &= \text{Mathematics} + \text{Programming} + \text{Physics} + \text{English} + \text{Electronics} \\ \text{Percentage} &= \text{Total Marks} / 5\end{aligned}$$

For example, if a student scores 82, 91, 76, 84 and 79, the total is 412 out of 500 and the percentage is 82.4%. The corresponding grade is A.

## 10. Algorithm

1. Start the program.
2. Display the main menu.
3. Ask the user to enter a choice.
4. If Add Student is selected, take student details and marks.
5. Validate the marks.
6. Store the student record.
7. If View Students is selected, display stored records.
8. If Search Student is selected, search using the roll number.
9. If Calculate Grade is selected, calculate total, percentage and grade.
10. If Exit is selected, stop the program.
11. Otherwise, display an invalid-choice message.
12. Repeat the menu until Exit is selected.

## 11. Python Concepts Used

- **Variables:** Used to store names, roll numbers and marks.
- **Lists:** Used to keep multiple student records.
- **Dictionaries:** Used to store information belonging to a student.
- **Functions:** Used to organize the program into a main function.
- **Conditional statements:** Used for menu choices, validation and grade calculation.
- **Loops:** Used to keep the menu running and to search/display records.
- **Arithmetic operations:** Used to calculate total and percentage.
- **Exception handling:** Used to handle invalid numeric input.

## 12. Sample Output

```
===== STUDENT GRADE MANAGEMENT SYSTEM =====

1. Add Student
2. View Students
3. Search Student
4. Calculate Student Grade
5. Exit

=====

Enter your choice: 1

----- ADD STUDENT -----

Enter roll number: 101
Enter student name: Rahul
Enter Mathematics marks: 82
Enter Programming marks: 91
Enter Physics marks: 76
Enter English marks: 84
Enter Electronics marks: 79

Student added successfully!
```

## 13. Testing

| Test Case | Input / Action                  | Expected Result            |
|-----------|---------------------------------|----------------------------|
| 1         | Valid student details and marks | Student added successfully |
| 2         | Marks greater than 100          | Invalid marks message      |
| 3         | Negative marks                  | Invalid marks message      |
| 4         | Existing roll number            | Student details displayed  |
| 5         | Unknown roll number             | Student not found          |
| 6         | Invalid menu choice             | Invalid choice message     |
| 7         | Exit option                     | Program closes             |

## 14. Limitations

- Student records are not permanently stored.
- There is no database.
- There is no graphical interface.
- There is no login system.
- The number of subjects is fixed.

## **15. Future Scope**

- Add file handling for permanent storage.
- Add update and delete options.
- Add more subjects.
- Add attendance information.
- Add database connectivity.
- Add a graphical user interface.
- Generate student reports.

## **16. Conclusion**

The Student Grade Management System demonstrates how basic Python programming concepts can be combined to create a useful command-line application. The project provides practice with input/output, variables, conditions, loops, lists, dictionaries, functions and basic error handling.

The project can also be extended in the future with permanent storage, additional features and a graphical interface.